

10주차 발표자료 요약본_최지희

[발표 1] Adam: A Method for Stochastic Optimization

1. 연구 배경 및 목적

딥러닝의 목적 함수(Loss function)는 본질적으로 확률적(Stochastic)인 특성을 가집니다. 전체 학습 데이터의 평균을 구하는 것은 비용이 크기 때문에 실제로는 미니배치를 통한 경험적 근사를 사용하며, 이 과정에서 매 스텝 그래디언트가 달라지는 노이즈가 발생합니다. 또한 Dropout과 같은 정규화 기법은 추가적인 노이즈를 주입하므로 이에 강건한 옵티마이저가 필수적입니다. 기존의 2차 미분(Hessian) 기반 방법은 수백만 개의 파라미터를 가진 모델에서 역행렬 계산 비용 $O(d^2 \sim d^3)$ 이 비현실적이기 때문에, 효율적이고 노이즈에 강건한 1차(First-order) stochastic 옵티마이저로서 Adam이 제안되었습니다.

2. 옵티마이저의 발전 계보와 Adam의 위치

옵티마이저는 크게 두 가지 방향으로 발전해 왔습니다.

- **방향성 개선:** 관성을 활용해 진동을 줄이는 Momentum(1964)과 이를 개선한 NAG(1983)가 있습니다.
- **스텝 사이즈 조절:** 파라미터별 학습률을 적응적으로 조절하는 AdaGrad(2011), 지수 이동 평균으로 학습률 소실 문제를 해결한 RMSProp(2012), 하이퍼파라미터를 제거하려는 AdaDelta(2012) 등이 있습니다.
- **Adam (2014):** Momentum의 방향 정보(1차 모멘트)와 RMSProp의 크기 정보(2차 모멘트)를 결합하여 방향과 스텝 사이즈를 동시에 적응적으로 조절하는 통합된 강점을 가집니다.

3. Adam 알고리즘의 핵심 메커니즘

Adam은 'Adaptive Moment Estimation'의 약자로, 두 가지 모멘트를 추정하여 파라미터를 업데이트합니다.

1. **그래디언트 계산:** 각 타임스텝 t 에서 미니배치 손실의 미분값 g_t 를 구합니다.
2. **모멘트 갱신:** 1차 모멘트 (그래디언트 평균)와 2차 모멘트 v_t (그래디언트 제곱 평균)를 지수 이동 평균(EMA) 방식으로 갱신합니다.
3. **편향 보정(Bias Correction):** $m_0 = 0, v_0 = 0$ 으로 초기화하기 때문에 초기 스텝에서 추정치가 0으로 편향되는 문제가 발생합니다. 이를 해결하기 위해 $(1 - \beta_1^t)$ 및

$(1 - \beta_2^t)$ 로 나누어 실제 그래디언트 값을 안정적으로 회복하도록 보정합니다.

4. **파라미터 업데이트:** 보정된 모멘트를 사용하여 파라미터를 갱신합니다. 기본 하이퍼파라미터 설정값은 $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$ 입니다.

4. SNR 해석 및 실효 변화량 상한

Adam의 업데이트 식 $\frac{\hat{m}_t}{\sqrt{\hat{v}_t}}$ 는 '신호 대 잡음비(SNR)'로 해석될 수 있습니다. 그래디언트가 일관되면(SNR 높음) 큰 스텝으로 빠르게 전진하고, 노이즈가 크거나 최적값 근처라면(SNR 낮음) 작은 스텝으로 자동 Annealing되는 효과를 가집니다. 또한, 실효 변화량에는 상한이 존재하여 학습이 발산하지 않도록 보장하며, α 는 파라미터 공간에서의 신뢰 구간 역할을 합니다.

5. 실험 결과 및 결론

Adam은 MNIST 분류(Dense feature), IMDB 리뷰(Sparse feature), MLP 및 CNN 등 다양한 환경에서 실험되었습니다. 특히 Sparse한 환경에서 AdaGrad 및 RMSProp과 함께 우수한 성능을 보였으며, CNN 실험에서는 단기와 장기 수렴 모두에서 가장 낮은 Cost에 도달함을 확인했습니다. 결론적으로 Adam은 효율성, 스케일 불변성, 직관적인 하이퍼파라미터라는 3대 강점을 바탕으로 현재 딥러닝 모델의 사실상 표준(De facto Standard) 옵티마이저로 자리 잡았습니다.

[발표 2] Efficient Memory Management for Large Language Model Serving with PagedAttention

1. 연구 배경: LLM 서빙의 병목 현상

LLM 서빙에서 처리량(Throughput) 향상은 요청당 비용을 낮추기 위한 핵심 과제입니다. LLM은 Autoregressive하게 토큰을 하나씩 생성하는데, 이때 이전 모든 토큰의 Key, Value 벡터를 저장하는 **KV Cache**가 필수적입니다. NVIDIA A100 GPU 메모리 분포를 보면 파라미터가 65%를 차지하고 KV Cache가 30% 이상을 차지하며, 서빙 과정이 메모리 대역폭에 제한을 받는(Memory-bound) 특징을 보입니다.

2. 기존 시스템의 한계점: 파편화와 공유 불가

기존 시스템(Faster Transformer, Orca 등)은 KV Cache를 연속된 메모리(Contiguous memory)에 저장하면서 두 가지 주요 비효율을 초래합니다.

- **Fragmentation:** 요청의 최대 길이에 맞춰 메모리를 미리 할당(Pre-allocation)하지만 실제 출력 길이는 짧은 경우가 많아 내부 파편화(Internal fragmentation)가 발생합니다. 또한 요청마다 할당 크기가 달라 외부 파편화도 발생하며, 실제 유효 메모리 비율은 20.4%~38.2% 수준에 불과합니다.

- **Memory Sharing 불가:** Parallel sampling이나 Beam search 같은 알고리즘은 시퀀스 간 KV Cache 공유가 가능함에도 불구하고 연속 메모리 구조 때문에 이를 활용하지 못합니다.

3. PagedAttention과 vLLM의 제안

본 논문은 운영체제의 가상 메모리 및 페이징 기법에서 영감을 받은 **PagedAttention** 알고리즘과 이를 활용한 서빙 엔진 **vLLM**을 제안합니다.

- **PagedAttention:** 각 시퀀스의 KV Cache를 고정 크기의 KV 블록으로 분할하여 불연속적인 물리 메모리 공간에 저장할 수 있도록 합니다.
- **KV Cache Manager:** 논리적 블록을 물리적 블록에 매핑하는 블록 테이블(Block table)을 관리합니다. 이를 통해 필요할 때만 블록을 동적으로 할당하여 메모리 낭비를 거의 제로(Near-zero waste)로 줄입니다.

4. 다양한 디코딩 시나리오로의 확장

vLLM은 블록 단위 관리를 통해 다양한 디코딩 상황에서 메모리 효율을 극대화합니다.

- **Parallel Sampling:** 공통 프롬프트의 KV Cache를 하나의 물리 블록에 저장한 뒤 참조 횟수(Reference count)를 기록하며 공유합니다. 시퀀스가 달라지는 부분만 Copy-on-Write(COW) 방식으로 처리합니다.
- **Beam Search:** 디코딩 진행에 따라 동적으로 변화하는 공유 패턴을 처리하여 빈번한 메모리 복사 오버헤드를 줄입니다.
- **Shared Prefix:** 시스템 프롬프트 등 자주 사용되는 프리픽스를 미리 물리 블록에 예약해두어 계산 중복을 제거합니다.

5. 스케줄링 및 분산 실행 전략

- **Scheduling:** FCFS 정책을 기본으로 하되, 메모리 부족 시 'All-or-nothing' 정책에 따라 특정 시퀀스의 블록 전체를 추출(Evict)합니다. 복구 방법으로는 CPU로 블록을 옮기는 Swapping과 다시 계산하는 Recomputation 중 효율적인 방식을 선택합니다.
- **Distributed Execution:** 대형 모델을 위해 Megatron-LM 방식의 Tensor model parallelism을 지원하며, 중앙 집중식 스케줄러 내 단일 KV Cache Manager가 여러 GPU Worker를 제어합니다.

6. 실험 결과 및 결론

OPT 및 LLAMA 모델을 사용한 실험 결과, vLLM은 기존의 Faster Transformer나 Orca 대비 **2~4배 높은 처리량**을 달성했습니다. 특히 시퀀스가 길고 모델이 크며 디코딩 알고리즘이 복잡할수록 개선 효과가 뚜렷했습니다. 블록 크기는 16일 때 하드웨어 활용도와 파편화

방지 사이의 균형이 가장 잘 맞았습니다. 결론적으로 PagedAttention은 OS의 성숙한 기법을 LLM 서빙에 성공적으로 이식하여 메모리 효율성을 획기적으로 개선한 연구입니다.